Google Stitch Core

Módulo 1: Naturaleza y Arquitectura Core

El sistema computacional denominado Stitch, concebido y operado dentro de la infraestructura de Google Labs, representa un cambio de paradigma en la automatización del desarrollo frontend, funcionando a nivel fundamental como un motor de transpilación multimodal impulsado por redes neuronales de gran escala.[1] A diferencia de los entornos de desarrollo integrados (IDE) tradicionales o las herramientas de diseño vectorial basadas en lienzos interactivos (canvas), la arquitectura de Stitch no manipula píxeles bidimensionales ni requiere la manipulación manual de vectores geométricos. En su lugar, opera mediante un bucle de orquestación algorítmica donde un modelo fundacional actúa como el analizador léxico, el motor de inferencia espacial y el compilador de código simultáneamente.[2] Este motor procesa intenciones de diseño abstractas—suministradas ya sea mediante lenguaje natural estructurado o representaciones visuales no estructuradas—y ejecuta una compilación determinista hacia un modelo de objetos de documento (DOM) válido, acoplado con reglas de estilo en cascada y una estructura jerárquica de componentes.[1] La naturaleza del sistema colapsa el ciclo de vida del desarrollo de software frontend tradicional, reemplazando la transferencia manual (handoff) entre los equipos de diseño de interfaces y los ingenieros de software mediante la síntesis directa de la topología visual y su correspondiente base de código renderizable.[2]

La topología de los modelos subyacentes que alimentan este motor de transpilación no es monolítica, sino que se encuentra bifurcada en dos entornos de procesamiento con características arquitectónicas y perfiles de rendimiento marcadamente distintos, documentados internamente como el "Standard Mode" y el "Experimental Mode".[1] Esta segmentación responde a la necesidad de gestionar las compensaciones inherentes entre la latencia de inferencia, la profundidad de la ventana de contexto, el consumo de memoria y la capacidad del modelo para ejecutar razonamiento espacial complejo sobre modalidades mixtas.[5]

El modo estándar (Standard Mode) está fundamentado en la arquitectura del modelo Gemini 2.5 Flash.[5] Desde una perspectiva de ingeniería de sistemas, este modelo está altamente optimizado para minimizar el tiempo hasta el primer token (Time-To-First-Token) y maximizar el rendimiento transaccional en entornos de alta concurrencia. Su función primaria es la transpilación estricta de Texto a Interfaz de Usuario (Text-to-UI).[5] En este modo, el sistema exhibe una latencia de inferencia considerablemente baja, permitiendo la generación de estructuras de interfaz completas en un marco temporal de sesenta a noventa segundos.[6] El menor consumo de parámetros de esta red neuronal permite optimizaciones computacionales específicas, siendo la más crítica la capacidad de ejecutar un mapeo bidireccional entre el árbol de sintaxis generado y los esquemas de archivos de herramientas de diseño externas. De manera documentada, esta topología facilita la traducción en memoria del diseño generado

hacia la especificación de auto-layout de Figma, exportando estructuras en capas editables y consistentes que los sistemas más pesados no logran compilar sin incurrir en cuellos de botella de memoria.[1]

Por el contrario, el modo experimental (Experimental Mode) delega su carga computacional a los modelos Gemini 2.5 Pro y a las iteraciones preliminares de la serie Gemini 3.0.[5] Esta arquitectura presenta una densidad de parámetros significativamente mayor, lo que incrementa el costo computacional, eleva la latencia general del sistema y dispara el consumo de tokens durante los ciclos de inferencia profunda.[5] Sin embargo, esta expansión en la capacidad de la red neuronal habilita la funcionalidad crítica de la transpilación de Imagen a Interfaz de Usuario (Image-to-UI).[8] Este modo incorpora codificadores de visión avanzados capaces de procesar tensores de imágenes que representan bocetos dibujados a mano, esquemas de baja fidelidad (wireframes) o capturas de pantalla de interfaces existentes.[8] A pesar de su potencia analítica, la complejidad de alinear las representaciones latentes de la visión con el código fuente introduce restricciones severas. Debido a la naturaleza estocástica del análisis espacial complejo, este modo suprime el determinismo necesario para la integración nativa con el motor de exportación de Figma, limitando la salida final exclusivamente a la capa de hipertexto y estilos.[1] Además, las documentaciones técnicas sobre la transición a la arquitectura Gemini 3.0 revelan vulnerabilidades en la ventana de contexto de salida, donde el sistema manifiesta una truncación agresiva de tokens al intentar reconstruir el código para topologías visuales excesivamente densas, lo que resulta en la pérdida de marcado y el colapso de la renderización en los nodos inferiores del árbol del diseño.[7]

El análisis del procesamiento de los vectores de entrada (Input Vectors) revela los mecanismos internos mediante los cuales el sistema transforma datos en bruto en estructuras de interfaz. En el canal de procesamiento Text-to-UI, el motor emplea técnicas de procesamiento de lenguaje natural para desensamblar el flujo de texto en un grafo de intenciones espaciales.[6] El analizador léxico subyacente identifica entidades sustantivas que se correlacionan con primitivas de interfaz, tales como paneles de navegación, tarjetas de perfil o rejillas de datos, al mismo tiempo que extrae modificadores adjetivales que definen la semántica de estado, como los esquemas de color, las tipografías deseadas o los comportamientos de espaciado.[2] Estos vectores de características se proyectan a través de capas de atención cruzada que los alinean con un espacio latente de conocimiento de diseño web, permitiendo al decodificador generar la estructura jerárquica pertinente.

En contraste, el canal Image-to-UI introduce una complejidad algorítmica exponencial mediante el uso de la visión por computadora.[8] Cuando el sistema recibe un arreglo de píxeles correspondiente a un boceto, el modelo de visión no realiza una simple traducción óptica de caracteres (OCR), sino una segmentación semántica profunda del lienzo bidimensional.[9] El sistema calcula cajas delimitadoras (bounding boxes) alrededor de formas ambiguas, clasifica las regiones demarcadas utilizando redes neuronales convolucionales o transformadores de visión (ViTs), y establece un grafo de dependencias espaciales que define las distancias relativas, los alineamientos ortogonales y las relaciones jerárquicas de contención entre los

elementos detectados.[9] Este grafo espacial abstracto se transfiere luego a un motor de reglas que lo proyecta hacia sistemas de diseño estandarizados, traduciendo una línea irregular dibujada en un papel en un contenedor Flexbox o CSS Grid matemáticamente preciso en el código final.[6]

Característica Técnica	**Standard Mode (Text-to-UI)**	**Experimental Mode (Image-to-UI)**
Motor de Inferencia Base	Gemini 2.5 Flash	Gemini 2.5 Pro / Gemini 3.0
Análisis de Input	Analizador léxico y atención semántica	Segmentación espacial y Transformadores de Visión
Capacidad de Exportación	Interoperabilidad dual (Código + Figma)	Singular (Exclusivamente compilación de Código)
Comportamiento de Tokens	Optimizado para contextos amplios y rápidos	Susceptible a truncación agresiva de salida (Gemini 3.0)
Latencia de Transpilación	Baja (Aproximadamente 60-90 segundos)	Alta (Dependiente de la complejidad topológica visual)

Módulo 2: Anatomía del Output y Límites del Sistema

La estructura de salida materializada por el motor de transpilación no se emite como un flujo aleatorio de caracteres, sino que es el producto final de la instanciación e iteración sobre un Árbol de Sintaxis Abstracta (AST).[10] Durante la fase de generación, el sistema primero modela el diseño conceptual como una estructura de datos jerárquica en memoria, separando lógicamente los componentes de contorno general (marcos de diseño) de los componentes terminales u hojas (accesorios e interacciones).[10] Esta representación en AST permite al motor validar sintácticamente las jerarquías anidadas antes de la serialización. Una vez que la estructura del árbol en memoria se estabiliza de acuerdo con la inferencia del vector de entrada, el decodificador transita el AST y compila la representación intermedia en lenguajes

de marcado estándar.[10]

El artefacto final expedido por la plataforma consiste estrictamente en un acoplamiento directo de HTML semántico y un modelo de CSS monolítico, predominantemente implementado a través de clases de utilidad inyectadas directamente en los nodos del DOM (como las generadas por arquitecturas similares a Tailwind CSS).[1] El motor produce esqueletos funcionales que mapean la disposición espacial del diseño de manera altamente predecible.[6] No obstante, la disección analítica de esta estructura estática revela debilidades arquitectónicas inmutables que definen los límites duros del sistema en entornos de producción. Al estar concebido como una herramienta generadora de prototipos puramente visuales, el código resultante exhibe deficiencias críticas en áreas que requieren conocimiento del tiempo de ejecución o estado subyacente.[1]

La primera carencia estructural profunda radica en la ausencia sistemática de atributos de accesibilidad para aplicaciones ricas de internet (ARIA).[1] El motor de transpilación, centrado exclusivamente en la representación gráfica de las cajas delimitadoras, omite la inferencia de semántica de accesibilidad en el AST. El código emitido no inyecta dinámicamente roles específicos de lector de pantalla (ej., role="dialog", role="navigation"), no genera identificadores únicos para la asociación de etiquetas abstractas (aria-labelledby), ni gestiona los índices de tabulación (tabindex) que definen el flujo lógico de navegación por teclado. Consecuentemente, el DOM generado es opaco para las tecnologías de asistencia, requiriendo que los ingenieros humanos refactoricen manualmente los árboles de nodos para inyectar la capa de semántica accesible omitida por el modelo.

La segunda limitación arquitectónica inmutable es la incapacidad del sistema para resolver el enrutamiento del lado del cliente (Client-Side Routing).[1] Aunque la interfaz de la plataforma introdujo funcionalidades de prototipado que permiten vincular lógicamente múltiples pantallas dentro del lienzo visual de la herramienta, esta conexión lógica no se transpila a controladores de rutas ejecutables en el código fuente.[6] El motor genera vistas monolíticas y aisladas que carecen de la lógica para interactuar con la API del Historial del navegador (History API) o gestionar la interceptación de eventos de navegación sin recarga de página. El código de salida no contiene rastros de arquitecturas de aplicaciones de una sola página (SPA), dejando la implementación de enrutadores basados en hash o en historial estructuralmente ausentes y dependientes de bibliotecas externas post-exportación.

La tercera y más restrictiva limitación técnica es la inexistencia de mecanismos de vinculación de datos (Data Binding) o manejo de estado dinámico.[1] El binomio de HTML y CSS emitido carece por completo de lógica imperativa o reactiva. Las representaciones de estados transitorios, tales como modales superpuestos, menús desplegables expandidos, o indicadores de carga de procesos asíncronos, se representan únicamente como estructuras de hipertexto rígidas con un estado visual congelado. El AST generado por el LLM no posee primitivas para declarar el estado local de un componente, pasar propiedades a través de jerarquías de componentes hijos (prop drilling), o establecer escuchadores de eventos del ciclo de vida (event listeners). Esta deficiencia implica que cualquier comportamiento dinámico

implícito en el diseño (por ejemplo, el filtrado de una tabla de datos) no existe en la carga útil exportada, debiendo ser programado retroactivamente utilizando ecosistemas externos como React, Vue o Angular.[6]

Estos límites estructurales de la arquitectura de software se ven complementados y forzados por límites de cuota y restricciones de red estipulados por el entorno de Google Labs. La plataforma implementa contramedidas de control de tráfico para evitar el agotamiento de los recursos computacionales de inferencia multimodal.[4]

Capa de Restricción	**Parámetro Técnico**	**Límite Documentado**	**Implicación en el Sistema**
Cuota de Transpilación	Standard Mode	350 generaciones por mes	Diseñado para habilitar búsquedas exploratorias y ajustes micro-tipográficos utilizando el modelo Flash de menor consumo.
Cuota de Inferencia	Experimental Mode	200 generaciones por mes	Restringe severamente el flujo de análisis por visión artificial (Image-to-UI), forzando convergencia temprana en los bocetos subidos.
Arquitectura de Red	Autenticación de Protocolo	Cierre de fallos por falta de principal	Rechazo total de claves de API genéricas; requiere obligatoriamente tokens de acceso OAuth2 basados en identidad o gcloud ADC.[12]

A nivel de red, las documentaciones de interoperabilidad establecen que la comunicación remota con los endpoints de la API (stitch.googleapis.com) no admite esquemas de

autenticación simples mediante claves de API estáticas (API Keys).[12] La arquitectura requiere inherentemente credenciales que afirmen una entidad de seguridad principal válida. Esta restricción técnica fuerza a cualquier puente de comunicación externo, como proxies locales o conectores de red de línea de comandos, a depender exclusivamente de los flujos de autorización OAuth2 o del marco de Credenciales Predeterminadas de la Aplicación (Application Default Credentials - ADC) gestionadas a través del SDK de Google Cloud. Cualquier carga útil enviada con credenciales portadoras no respaldadas por una identidad autenticada desencadena un rechazo inmediato por parte de la puerta de enlace, interrumpiendo las tuberías de orquestación de datos.[12]

Módulo 3: Interfaz y Manipulación de Estado

El procesamiento iterativo de estructuras de interfaces mediante modelos de lenguaje expone una vulnerabilidad fundamental en las arquitecturas basadas en mecanismos de atención: la degradación o sangrado del contexto computacional, comúnmente denominado "Context Bleed".[3] En el transcurso de una sesión prolongada, a medida que un usuario introduce múltiples instrucciones paramétricas aditivas (por ejemplo, solicitar la inyección de filtros de búsqueda, seguida de modificaciones iconográficas y, posteriormente, alteraciones en el esquema de paletas), la matriz de auto-atención del transformador pierde coherencia sobre los nodos iniciales del diseño.[14] Este desbordamiento de información provoca una inestabilidad sistémica en la que el generador descarta subárboles enteros previamente validados, optando por una regeneración catastrófica que destruye la continuidad espacial y estilística del artefacto.[14]

Para contrarrestar y resolver técnicamente el fenómeno del Context Bleed, la plataforma incorpora internamente un parámetro de control de estado denominado herramienta "Multi-select".[15] Lejos de ser una simple comodidad de la interfaz de usuario, el mecanismo Multi-select funciona como un enrutador de enmascaramiento sobre los tensores de inferencia del modelo subyacente. Cuando se efectúa una selección múltiple sobre la estructura visual, el sistema no reenvía la totalidad del AST al motor de transpilación para su reevaluación completa. En su lugar, el sistema calcula máscaras delimitadoras precisas en el espacio de representación latente.[17] Esta lógica congela los gradientes y bloquea los pesos de todos los nodos del árbol DOM que caen fuera de las cajas delimitadoras seleccionadas. En consecuencia, el vector de atención de la red neuronal se confina espacialmente y restringe sus ciclos de inferencia de modificación de hipertexto exclusivamente a los fragmentos del AST seleccionados.[17] Esta vectorización focalizada mitiga drásticamente la amnesia iterativa del sistema, garantizando que el resto de la interfaz permanezca inmutable mientras se procesan mutaciones locales de alta densidad lógica.

El mantenimiento de la integridad estructural a lo largo de las sesiones de diseño introduce un segundo desafío arquitectónico: la lógica de versionado. El sistema rechaza explícitamente el paradigma convencional de historial lineal (una simple pila de acciones deshacer/rehacer) en favor de un sistema de persistencia modelado como un Grafo Acíclico Dirigido (DAG - Directed

Acyclic Graph).[18] Este cambio topológico en el manejo del estado del diseño significa que las iteraciones en el lienzo de trabajo operan bajo principios similares a los sistemas de control de versiones descentralizados.

En el marco del DAG de Stitch, cada llamada de transpilación exitosa que altera la interfaz no sobrescribe los bloques de memoria del estado anterior, sino que genera un nuevo vértice (nodo inmutable) en el grafo.[20]

1. **Vértices Aislados:** Cada nodo consolida y encapsula la totalidad del estado del diseño en un instante temporal específico, manteniendo la serialización completa de la sintaxis y los parámetros estilísticos.
2. **Aristas Transaccionales (Edges):** Los vínculos causales que conectan un diseño base con su diseño modificado se gestionan mediante aristas direccionales, que operan como punteros referenciales documentando exactamente qué vector de transformación se aplicó.
3. **Bifurcación Múltiple (Branching):** La topología acíclica permite la bifurcación asíncrona del estado del diseño sin desencadenar colisiones destructivas.[22] Un único nodo base puede ramificarse en docenas de variantes experimentales en paralelo (por ejemplo, evaluaciones comparativas de topologías de navegación). La estructura matemática del DAG asegura que el historial transaccional permanezca absoluto, proporcionando tiempos de recuperación de estados anteriores altamente eficientes y asegurando la auditabilidad de la evolución del componente generativo a lo largo de su ciclo de vida computacional.[21]

La encapsulación del estado del diseño interactúa de manera directa con las políticas de aislamiento de entorno y privacidad de telemetría a nivel del clúster de servidores de Google Labs. El sistema expone un parámetro técnico crítico dentro de la configuración de sesión: el estado booleano de exclusión de telemetría definido como Allow AI model training = false.[4] La conmutación de este parámetro altera inmediatamente la trayectoria de los datos generativos dentro de la infraestructura backend. Cuando este interruptor lógico se desactiva, el sistema instruye a la capa de persistencia de datos a marcar todos los metadatos de transpilación entrantes, las estructuras AST generadas, y las representaciones tensoriales de las solicitudes de lenguaje natural o fragmentos de imagen con un protocolo de aislamiento de entorno.[4] Internamente, los registros de datos asumen un indicador de tiempo de vida (TTL) estrictamente efímero y son físicamente excluidos de las tuberías de ingesta que alimentan los conjuntos de datos de ajuste fino (fine-tuning) de los modelos fundacionales masivos de Google.[4] El impacto en la sesión es que se instaura una barrera arquitectónica (sandbox) que previene el sangrado de propiedad intelectual estructural, asegurando que patrones de diseño cerrados o lógicas de negocio patentadas transmitidas en los vectores de entrada permanezcan inescrutables para los procesos algorítmicos asíncronos de reentrenamiento a nivel global.

Módulo 4: Ecosistema MCP e Interoperabilidad

La capacidad de Stitch para trascender los confines de su interfaz web encapsulada e integrarse profundamente con las arquitecturas de compilación de desarrollo locales depende de su interconexión mediante el Protocolo de Contexto de Modelos (MCP - Model Context Protocol). La capa de transporte fundamental de esta integración está gestionada por el paquete de representación y comunicación distribuido bajo el espacio de nombres @_davideast/stitch-mcp.[13] Esta infraestructura actúa como un proxy inverso de autorización y un enrutador de Llamada a Procedimiento Remoto (RPC) que permite a sistemas locales no autenticados interactuar programáticamente con el entorno remoto protegido.[12]

La arquitectura del proxy stitch-mcp aborda un cuello de botella de seguridad sustancial: la incapacidad de los agentes de codificación autónomos locales (sistemas como Antigravity, entornos en editores como Cursor, o agentes basados en CLI) para negociar flujos de identidad en la nube de Google durante tiempos de ejecución aislados.[12] El proxy elude esta fricción estableciendo una sesión multiplexada y persistente en el sistema del usuario mediante flujos de entrada/salida estándar (STDIO), enlazándose contra el endpoint backend principal stitch.googleapis.com utilizando las Credenciales Predeterminadas de la Aplicación del entorno (ADC) y protocolos gcloud en segundo plano.[12] Una vez que el conducto TLS bidireccional se inicializa y se verifica, el daemon local traduce de forma transparente las llamadas de herramientas abstractas (tool calls) del estándar MCP en operaciones RESTful validadas. Esto convierte al flujo de trabajo del proxy en un túnel de estado, habilitando a un agente de código que reside en una máquina macOS o Linux local para leer esquemas de estilo, volcar código fuente y extraer capturas de pantalla desde el repositorio de diseño alojado en la nube como si fuesen archivos de sistema de archivos locales.[25]

Para facilitar el control granular sobre esta arquitectura de proxy, el sistema expone un inventario estricto de comandos a través de su interfaz de línea de comandos (CLI), dividiendo su utilidad operativa en tres dominios lógicos: Configuración, Integración y Construcción.[13]

Comando CLI	**Familia Operativa**	**Comportamiento del Sistema y Función Técnica**
init	Setup	Ejecuta la inicialización de estado del entorno. Negocia los flujos de autorización con el proveedor OAuth de GCP, establece el ID del proyecto dentro de los descriptores de entorno y escribe el manifiesto de configuración del cliente MCP en el

		directorio local, aprovisionando la confianza inicial.[13]
doctor	Setup	Script de telemetría de red. Realiza sondeos y comprobaciones de integridad computacional contra la configuración local, evaluando la validez de los tokens, la accesibilidad de los contenedores de proyectos en stitch.googleapis.com y detectando divergencias en la matriz de permisos IAM.[13]
proxy	Integration	Instancia el bucle de eventos asíncrono primario. Este comando activa el transporte MCP sobre descriptores de archivo stdout/stdin, exponiendo los nodos virtuales remotos hacia los agentes locales que se comunican utilizando el formato JSON-RPC 2.0 estándar.[12]
serve	Development	Arranca un servidor HTTP efímero (-p <id>). Transforma el esquema del proyecto alojado en memoria en representaciones DOM que son servidas a través de un socket de red local para la introspección visual independiente mediante agentes de navegación web o humanos.[24]

view	Development	Lanza un explorador de recursos interactivo basado en terminal. Interroga el backend del proyecto en tiempo real, permitiendo la navegación en profundidad por la topología anidada de datos, la previsualización del HTML inyectado localmente y el acceso directo al portapapeles de los tokens extraídos.[13]
tool	Integration	Mecanismo de paso (passthrough) para el enrutamiento directo. Permite la invocación sincrónica de las herramientas del servidor MCP desde el shell del host eludiendo el protocolo de mensajería del agente. El sufijo -s fuerza la emisión del esquema de metadatos JSON requerido por la herramienta específica.[13]

Dentro de la arquitectura asíncrona de MCP, las operaciones atómicas sobre el código se exponen como herramientas (Tools) o nodos virtuales.[13] Un agente o cliente requiere conocer la estructura precisa de los datos de inicialización para consumir estas primitivas. El nodo fundamental para la materialización de componentes de diseño en arquitecturas locales se denomina build_site (a veces estructurado bajo la firma alias site en la CLI).[24]

El esquema JSON estructural requerido por el protocolo para activar el endpoint build_site debe conformarse estrictamente a las definiciones de validación de entradas preestablecidas en el servidor proxy. La carga útil no solo contiene referencias unívocas, sino que actúa como el contrato de serialización, determinando el nivel de extracción de dependencias y el pre-procesamiento de frameworks.[24] La estructura formalizada subyacente de la carga útil (payload) requiere típicamente la especificación del identificador universal del sistema Stitch, los descriptores del entorno de salida y las banderas de tratamiento de activos:

JSON

```
{
  "jsonrpc": "2.0",
  "method": "tools/call",
  "params": {
    "name": "build_site",
    "arguments": {
      "project_id": "string (Identificador unívoco del nodo raíz del DAG de Stitch)",
      "target_framework": "string (Define la estrategia de transpilación del AST, ej. 'astro', 'react')",
      "output_directory": "string (Ruta de acceso canónica del host local para el vertido del buffer)",
      "extract_assets": "boolean (Activa la descarga sincrónica de binarios y firmas visuales asociadas a los nodos)"
    }
  },
  "id": "number/string"
}
```

La ejecución de este nodo a través del proxy desencadena una secuencia en la cual el backend extrae el estado de la hoja actual del grafo del proyecto (DAG), transpila el HTML/CSS genérico hacia representaciones modulares compatibles con el entorno destino, y emite secuencias de bytes al directorio especificado.[24] El sistema implementa una estricta política "Fail-Closed" en estas operaciones [27]; si el flujo de descarga de paquetes experimenta inestabilidad, o si la validación estructural AST local falla en encontrar rutas válidas para dependencias (LSP trace errors), el servidor finaliza la ejecución asíncrona inmediatamente y expone el rastro de la pila (stack trace) nativo, impidiendo activamente que el agente llene las discrepancias con código alucinado no determinista.[27]

Módulo 5: Repositorio Oficial stitch-skills

Para aumentar significativamente la capacidad algorítmica y la precisión determinística de los agentes locales, Google Labs expone el código base google-labs-code/stitch-skills.[26] Este sistema no funciona como una biblioteca estática de utilidades importables estándar, sino que fundamenta una arquitectura de inyección de conocimiento cognitivo referida como el paradigma de "Agent Skills".[26]

A nivel computacional, el paradigma de Agent Skills aborda el problema de la excesiva entropía en el comportamiento autónomo de los LLM (como Gemini CLI o Claude Code) al interactuar con código de UI.[26] En lugar de permitir que el agente infiera el contexto arquitectónico de un proyecto a partir de simples instrucciones (prompts), el comando CLI de instalación de Skills inyecta un andamiaje pre-estructurado de reglas, validaciones de sintaxis abstracta y flujos de enrutamiento asíncronos en el disco del entorno local.[26] La estructura canónica impuesta a

cada módulo instalado fuerza un protocolo determinista mediante múltiples subdirectorios. El núcleo operativo reside en el manifiesto SKILL.md, que actúa como la instrucción fundacional de control de misión del LLM; el directorio scripts/, que contiene rutinas computacionales inflexibles que imponen la validación topológica y de red; y los directorios resources/ y examples/, que aprovisionan las técnicas de aprendizaje de pocos disparos (few-shot prompting) y la ontología de validación requerida para mantener las transiciones de estado del agente puras.[26]

El catálogo estructural expuesto por el repositorio se compone de módulos funcionales especializados, donde cada instancia ejecuta operaciones de procesamiento semántico, transpilación AST o manipulaciones de estado de archivo predefinidas en su lógica interna de backend.[26]

Módulo Interno	**Categoría de Operación**	**Descripción Técnica y Mecanismo Backend**
stitch-design	Enrutamiento e Inyección Semántica	Actúa como el orquestador principal del ciclo de transpilación.[28] Su cálculo evalúa condicionalmente el contexto del input (ej., tipo de dispositivo, plataforma objetivo) y, en caso de estar ausente, inyecta palabras clave estructurales precisas en la cadena generativa. Orquesta las llamadas API al servidor MCP para iniciar o actualizar la topología base del proyecto en la nube.[30]
design-md	Inferencia de Estado y Cristalización	Ejecuta operaciones asíncronas de lectura (GET) sobre el estado del proyecto a través de la red.[28] Extrae las ramificaciones estilísticas, los diccionarios de color y las métricas espaciales

		tipográficas del árbol de Stitch. El script backend realiza ingeniería inversa de la intención espacial y la compila en un documento estático de Markdown (.stitch/DESIGN.md) en la memoria local.[28] Este artefacto sirve como base de datos léxica para prevenir el "design token drift", bloqueando al agente en una fuente única de verdad estilística en cálculos posteriores.[26]
stitch-loop	Máquina de Estados de Transpilación	Implementa un bucle de construcción cíclico automatizado.[28] Funciona como un motor de orquestación que disgrega un aviso singular en un grafo de dependencias multipágina. El script en backend repite los ciclos de llamadas de síntesis a Stitch por cada página enumerada en el grafo, descargando, verificando y estructurando topológicamente el código resultante en directorios de un marco subyacente sin interrupción manual, habilitando la generación desatendida de plataformas web completas.[26]
react-components	Análisis AST y Manipulación de Nodos	El módulo de transpilación secundaria.[28] Recupera la carga estática de HTML empleando transferencias

		HTTP de red estables (ej., curl del sistema).[34] Inicializa un analizador de sintaxis abstracta en un proceso en segundo plano de Node.js que escanea el bloque HTML devuelto, realiza el seguimiento lógico de componentes de contenedores fijos, y muta el código hacia funciones de renderizado acopladas a la sintaxis JSX/TSX. Finalmente, dispara una rutina estricta de validación local (npm run validate <file_path>).[28] Si el árbol DOM de React devuelto es defectuoso o los tokens de estilo son inalcanzables, la validación falla y rechaza el artefacto antes de comprometerlo en el disco principal del proyecto.[28]
enhance-prompt	Ingeniería de Características NLP	Se enfoca en la calibración probabilística paramétrica. El script backend canaliza un string de alta entropía a través de múltiples transformadores.[28] Detecta heurísticamente descriptores visuales ambiguos y aplica un sistema de reemplazo mediante expresiones regulares o matrices condicionales, inyectando terminología de componentes de UI fuertemente tipados.[30] El

		resultado es una carga útil reorganizada matemáticamente y numerada secuencialmente que garantiza activaciones en sub-vectores más precisos del modelo subyacente de Stitch.[30]
shadcn-ui	Resolución de Dependencias de Terceros	Actúa como un proxy de mapeo lógico local. Escanea los elementos anidados complejos (ej., listas de opciones, campos interactivos o modales superpuestos) generados por el motor visual e interfiere con los cálculos de renderizado.[28] Cuando un agente intenta mapear estas vistas estáticas en React, el script bloquea construcciones personalizadas propensas a errores y ejecuta rutinas del entorno CLI (npx shadcn...) para inyectar componentes estandarizados del ecosistema nativo, forzando la conciliación estructural sin sobrescribir las clases de contorno o colores predeterminadas en el documento fundamental de estilo.[26]

Obras citadas

1. Design Mobile App UI with Google Stitch (Step-by-Step Guide) | Codecademy, fecha de acceso: marzo 15, 2026, https://www.codecademy.com/article/google-stitch-tutorial-ai-powered-ui-design-tool
2. UI Automation is Here: My Experience with Google's AI Design Tool, Stitch -

Medium, fecha de acceso: marzo 15, 2026, https://medium.com/google-cloud/ui-automation-is-here-my-experience-with-googles-ai-design-tool-stitch-7e59405ecc56
3. AI Fire Daily - Rss, fecha de acceso: marzo 15, 2026, https://media.rss.com/ai-fire-daily/feed.xml
4. Google Stitch AI Walkthrough: Comparison, Features, Alternatives and Pricing - UX Pilot, fecha de acceso: marzo 15, 2026, https://uxpilot.ai/blogs/google-stitch-ai
5. Google Stitch Review 2026: AI UI Design Tool Features, Pros & Cons | Index.dev, fecha de acceso: marzo 15, 2026, https://www.index.dev/blog/google-stitch-ai-review-for-ui-designers
6. Google Stitch Build and Automate ANYTHING! : r/AISEOInsider - Reddit, fecha de acceso: marzo 15, 2026, https://www.reddit.com/r/AISEOInsider/comments/1ran6t9/google_stitch_build_and_automate_anything/
7. Gemini 3.1 Pro - Hacker News, fecha de acceso: marzo 15, 2026, https://news.ycombinator.com/item?id=47074735
8. Google Stitch: The Complete Guide to AI-Powered UI Design That's Revolutionizing App Development in 2026 | ALM Corp, fecha de acceso: marzo 15, 2026, https://almcorp.com/blog/google-stitch-complete-guide-ai-ui-design-tool-2026/
9. What Is Google Stitch? A Beginner's Guide (2026) - UI Things, fecha de acceso: marzo 15, 2026, https://uithings.com/what-is-google-stitch
10. Google Labs: The Experimental Playground I've Been Ignoring for Way Too Long - Medium, fecha de acceso: marzo 15, 2026, https://medium.com/@gokulofficial18602/google-labs-the-experimental-playground-ive-been-ignoring-for-way-too-long-3324eff763e0
11. Daily Papers - Hugging Face, fecha de acceso: marzo 15, 2026, https://huggingface.co/papers?q=tree-structured%20multimodal%20code%20generation
12. Stitch MCP not working with API key - Google AI Developers Forum, fecha de acceso: marzo 15, 2026, https://discuss.ai.google.dev/t/stitch-mcp-not-working-with-api-key/120933
13. @_davideast/stitch-mcp - npm, fecha de acceso: marzo 15, 2026, https://www.npmjs.com/package/%40_davideast%2Fstitch-mcp
14. Stitch Prompt Guide - Google AI Developers Forum, fecha de acceso: marzo 15, 2026, https://discuss.ai.google.dev/t/stitch-prompt-guide/83844
15. pawamoy/stars: My stars on GitHub, grouped by language., fecha de acceso: marzo 15, 2026, https://github.com/pawamoy/stars
16. What's New Archives - Fusion Blog - Autodesk, fecha de acceso: marzo 15, 2026, https://www.autodesk.com/products/fusion-360/blog/tag/whats-new/feed/
17. ComfyUI Extensions - Comfy.ICU, fecha de acceso: marzo 15, 2026, https://comfy.icu/extension/
18. The Best ETL Tools for Salesforce in 2026: A Decision Framework - Definite, fecha de acceso: marzo 15, 2026, https://www.definite.app/blog/best-etl-tools-for-salesforce-2026

19. Towards Scientific Intelligence: A Survey of LLM-based Scientific Agents - arXiv, fecha de acceso: marzo 15, 2026, https://arxiv.org/html/2503.24047v3
20. Awesome Data Catalogs and Observability Platforms. - GitHub, fecha de acceso: marzo 15, 2026, https://github.com/opendatadiscovery/awesome-data-catalogs
21. COSMOS: Python library for massively parallel workflows - PMC, fecha de acceso: marzo 15, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC4184253/
22. US20210018953A1 - Asynchronous distributed coordination and consensus with threshold logical clocks - Google Patents, fecha de acceso: marzo 15, 2026, https://patents.google.com/patent/US20210018953A1/en
23. Track: Poster Session 6 - ICLR 2026, fecha de acceso: marzo 15, 2026, https://iclr.cc/virtual/2024/session/19811
24. davideast/stitch-mcp: A CLI for moving AI-generated UI ... - GitHub, fecha de acceso: marzo 15, 2026, https://github.com/davideast/stitch-mcp
25. Codvyn Antigravity Claudecode Stitch | PDF | Command Line Interface - Scribd, fecha de acceso: marzo 15, 2026, https://www.scribd.com/document/1004246759/Codvyn-Antigravity-Claudecode-Stitch
26. Stitch Agent Skills repo for design-to-code automation : r/StitchAI - Reddit, fecha de acceso: marzo 15, 2026, https://www.reddit.com/r/StitchAI/comments/1r1yf7a/stitch_agent_skills_repo_for_designtocode/
27. i bet you might not get better output without this : r/google_antigravity - Reddit, fecha de acceso: marzo 15, 2026, https://www.reddit.com/r/google_antigravity/comments/1rdl85o/i_bet_you_might_not_get_better_output_without_this/
28. google-labs-code/stitch-skills · GitHub - GitHub, fecha de acceso: marzo 15, 2026, https://github.com/google-labs-code/stitch-skills
29. r/StitchAI - Reddit, fecha de acceso: marzo 15, 2026, https://www.reddit.com/r/StitchAI/
30. stitch-design | Skills Marketplace - LobeHub, fecha de acceso: marzo 15, 2026, https://lobehub.com/pt-BR/skills/practicalswan-agent-skills-stitch-design
31. How to Maintain Style Consistency - Stitch - Google AI Developers Forum, fecha de acceso: marzo 15, 2026, https://discuss.ai.google.dev/t/how-to-maintain-style-consistency/116160
32. From Prompt to Product: Investigating AI-Driven Web Design with Antigravity + Stitch, fecha de acceso: marzo 15, 2026, https://scuti.asia/from-prompt-to-product-investigating-ai-driven-web-design-with-antigravity-stitch/
33. react:components | Skills Marketplace - LobeHub, fecha de acceso: marzo 15, 2026, https://lobehub.com/ru/skills/google-labs-code-stitch-skills-react-components
34. stitch-design | Skills Marketplace - LobeHub, fecha de acceso: marzo 15, 2026, https://lobehub.com/it/skills/practicalswan-agent-skills-stitch-design
35. Explore Skills - AI DevKit, fecha de acceso: marzo 15, 2026, https://ai-devkit.com/skills/